

# Ejemplo 3 (XMLHttpRequest + Promesa): Obtener Comentarios

## Objetivo del ejercicio

- Practicar **peticiones AJAX clásicas** con `XMLHttpRequest` usando **promesas**.
- Mostrar cómo manejar la respuesta con `.then()`, errores con `.catch()` y finalización con `.finally()`.
- Visualizar comentarios en **tarjetas de Bootstrap**, incluyendo nombre, email y contenido.

---

## Código completo (HTML + Bootstrap + JS)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo 3 - XMLHttpRequest con Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1 class="mb-4">Ejemplo 3: Obtener Comentarios con XMLHttpRequest
y Promesas</h1>
  <p>Haz click en el botón para cargar los primeros 10 comentarios
de la API pública <code>JSONPlaceholder</code>.</p>

  <button id="btnCargarComentarios" class="btn btn-warning">Cargar
Comentarios</button>

  <div id="resultadoComentarios" class="mt-3"></div>

  <script>
    const btnCargarComentarios =
document.getElementById('btnCargarComentarios');
    const resultadoComentarios =
document.getElementById('resultadoComentarios');

    // Función que devuelve una promesa con XMLHttpRequest para
comentarios
    function obtenerComentarios() {
      return new Promise((resolve, reject) => {
        const xhr = new XMLHttpRequest();
        xhr.open('GET',
'https://jsonplaceholder.typicode.com/comments', true);

        xhr.onreadystatechange = () => {
          if (xhr.readyState === 4) {
            if (xhr.status === 200) {
              const comentarios =
JSON.parse(xhr.responseText);
```

```

        resolve(comentarios);
    } else {
        reject(`Error ${xhr.status}:
    }
    }
    };

    xhr.send();
});
}

// Evento click para cargar comentarios
btnCargarComentarios.addEventListener('click', () => {
    resultadoComentarios.innerHTML = `<div class="alert alert-
info">Cargando comentarios...</div>`;

    obtenerComentarios()
        .then(comentarios => {
            let html = '';
            // Mostrar solo los primeros 10 comentarios
            comentarios.slice(0, 10).forEach(comment => {
                html += `
                <div class="card mb-3">
                    <div class="card-header">Comentario
#${comment.id} - Post ${comment.postId}</div>
                    <div class="card-body">
                        <h5 class="card-
title">${comment.name}</h5>
                        <p class="card-
text">${comment.body}</p>
                        <p class="text-muted">Email:
${comment.email}</p>
                    </div>
                </div>`;
            });
            resultadoComentarios.innerHTML = html;
        })
        .catch(error => {
            resultadoComentarios.innerHTML = `<div
class="alert alert-danger">${error}</div>`;
        })
        .finally(() => {
            console.log("Petición de comentarios finalizada");
        });
    });
</script>

</body>
</html>

```

---

## Explicación paso a paso

1. **Función obtenerComentarios():**
  - o Crea y devuelve una **promesa** que envuelve la petición AJAX clásica.
  - o `resolve(comentarios)` se llama cuando la petición es exitosa.
  - o `reject(error)` se llama si hay fallo (código HTTP distinto de 200).
2. **Evento click:**

- Llama a `obtenerComentarios()`.
  - `.then()` recibe los comentarios y los muestra en **tarjetas Bootstrap**.
  - `.catch()` captura cualquier error.
  - `.finally()` se ejecuta siempre, ideal para logs o limpieza.
3. **Procesamiento de datos:**
- Convertimos el JSON de la respuesta a objeto JS con `JSON.parse()`.
  - Mostramos solo los primeros 10 comentarios para simplificar la vista.
  - Cada tarjeta muestra: `id`, `postId`, `name`, `body` y `email`.
4. **Ventaja:**
- Mantiene compatibilidad con **XMLHttpRequest clásico**, pero con la sintaxis moderna de **promesas**.
- 

## Preguntas de comprensión para alumnos

1. ¿Qué hace `resolve()` y `reject()` dentro de la promesa?
  2. ¿Por qué se usa `slice(0, 10)` al mostrar los comentarios?
  3. ¿Cuál es la diferencia entre `.then()` y `.catch()`?
  4. ¿Cómo podrías mostrar todos los comentarios sin límite?
  5. ¿Qué otros elementos de los comentarios se podrían mostrar para enriquecer la interfaz?
- 

## Mejoras sugeridas para el alumno

1. Implementar **paginación** para mostrar más comentarios de forma organizada.
2. Añadir un **campo de búsqueda por nombre o email** del comentarista.
3. Mostrar los comentarios en **modal** al hacer click para ver detalles.
4. Añadir **spinner o animación de carga** mientras se realiza la petición.
5. Crear una función **reutilizable** que permita hacer peticiones AJAX con promesas para otras APIs.